AI for IoT - Laboratory Practical

Practical No. 2

Title

Data Collection from IoT Sensors and Preprocessing using Python.

Aim

To collect (simulate) data from IoT sensors using Python and to preprocess the collected raw sensor data handling missing values, removing duplicates and outliers, and normalizing the data so that it is ready for further analysis or machine learning tasks.

Apparatus / Requirements (Software-Based Experiment)

As this is a software-only practical, no physical IoT hardware (sensors/microcontrollers) is required.

A simulated dataset generated in Python is used to represent real IoT sensor readings.

Operating System	Windows / Linux / macOS
Programming Language	Python 3.x
IDE / Platform	Jupyter Notebook, Google Colab, or VS Code
Libraries Required	pandas, numpy, matplotlib, scikit-learn
Dataset	Simulated IoT sensor data (temperature, humidity, timestamp) generated in Python

Theory

IoT (Internet of Things) systems consist of sensor nodes that continuously generate data such as temperature, humidity, pressure, or motion readings. This raw sensor data is often noisy, irregular, and contains missing or duplicate values because of network drops, sensor faults, or transmission errors. Before this data can be used

for analysis, visualization, or feeding into an AI/ML model, it must be preprocessed.

Data preprocessing for IoT data typically involves:

- Data collection — acquiring readings from sensors (or, in this simulation, generating synthetic readings) and storing them, usually as a CSV/JSON stream.
- Data cleaning — identifying and handling missing values and removing duplicate records.
- Outlier detection — identifying sensor readings that are physically unrealistic or statistically abnormal.
- Normalization / scaling — rescaling numeric features to a common range (e.g., 0–1) so that no single feature dominates a downstream ML model.
- Visualization — plotting the cleaned data to verify trends and confirm the data is analysis-ready.

Procedure

1. Open Jupyter Notebook / Google Colab / VS Code and create a new Python (.ipynb or .py) file.
2. Import the required Python libraries — pandas, numpy, matplotlib, and scikit-learn.
3. Simulate IoT sensor data: generate a series of timestamps and corresponding random temperature and humidity readings using numpy, to mimic values that would normally arrive from physical sensors.
4. Deliberately insert a few missing values (NaN) and a duplicate row into the simulated dataset, to mimic real-world sensor faults.
5. Save the simulated raw readings to a CSV file (sensor_data.csv) — this represents the 'data collection' step of an IoT pipeline.
6. Load the CSV file back into a pandas DataFrame and inspect it (shape, column data types, first few rows, and count of missing values).
7. Handle missing values using mean/median imputation (or drop rows, depending on the use case).
8. Remove duplicate rows and detect outliers (e.g., using the IQR method or a simple threshold check).

9. Normalize the numeric sensor columns using Min-Max Scaling (or StandardScaler) from scikit-learn.
10. Visualize the cleaned, normalized data using matplotlib (line plot of readings over time).
11. Save the final preprocessed data to a new CSV file (preprocessed_data.csv).

Python Program

Step 1–5: Simulate and collect IoT sensor data

```
import numpy as np
import pandas as pd

np.random.seed(42)

# Simulate 50 sensor readings, one per minute
timestamps = pd.date_range('2026-07-17 09:00', periods=50, freq='T')
temperature = np.random.normal(loc=30, scale=2, size=50)   # deg C
humidity    = np.random.normal(loc=60, scale=5, size=50)   # % RH

sensor_data = pd.DataFrame({
    'timestamp': timestamps,
    'temperature': temperature,
    'humidity': humidity
})

# Inject a few missing values and a duplicate row to mimic real sensor
faults
sensor_data.loc[5, 'temperature'] = np.nan
sensor_data.loc[12, 'humidity'] = np.nan
sensor_data.loc[20, 'temperature'] = np.nan
sensor_data = pd.concat([sensor_data, sensor_data.iloc[[10]]],
ignore_index=True)

# Save raw collected data (simulates data arriving from the IoT gateway)
sensor_data.to_csv('sensor_data.csv', index=False)
print(sensor_data.head())
```

Step 6–9: Load and preprocess the data

```
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv('sensor_data.csv')
print('Shape:', df.shape)
print(df.isnull().sum())

# Handle missing values using column-wise mean imputation
df['temperature'] = df['temperature'].fillna(df['temperature'].mean())
df['humidity']    = df['humidity'].fillna(df['humidity'].mean())

# Remove duplicate rows
df = df.drop_duplicates().reset_index(drop=True)

# Simple outlier check using the IQR method (temperature column)
Q1, Q3 = df['temperature'].quantile([0.25, 0.75])
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
df = df[(df['temperature'] >= lower) & (df['temperature'] <= upper)]

# Normalize numeric columns to a 0-1 range
scaler = MinMaxScaler()
df[['temperature', 'humidity']] = scaler.fit_transform(df[['temperature',
'humidity']])
```

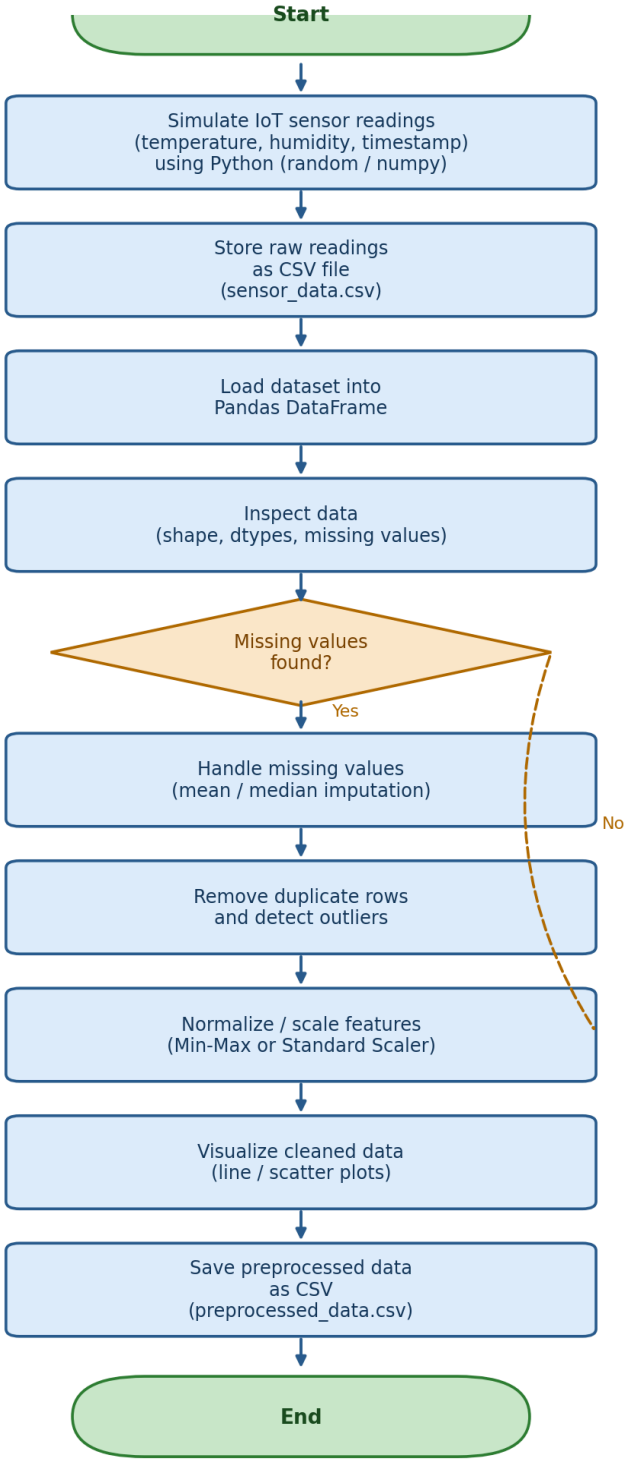
Step 10–11: Visualize and save the preprocessed data

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 4))
plt.plot(df['temperature'], label='Temperature (normalized)')
plt.plot(df['humidity'], label='Humidity (normalized)')
plt.xlabel('Reading Index')
plt.ylabel('Normalized Value')
plt.title('Preprocessed IoT Sensor Data')
plt.legend()
plt.tight_layout()
plt.savefig('preprocessed_plot.png')
plt.show()

df.to_csv('preprocessed_data.csv', index=False)
print('Preprocessing complete. File saved as preprocessed_data.csv')
```

Flowchart


Start
Simulate IoT sensor readings (temperature, humidity, timestamp) using Python (random / numpy)
Store raw readings as CSV file (sensor_data.csv)
Load dataset into Pandas DataFrame
Inspect data (shape, dtypes, missing values)
Missing values found?
Yes
No
Handle missing values (mean / median imputation)
Remove duplicate rows and detect outliers
Normalize / scale features (Min-Max or Standard Scaler)
Visualize cleaned data (line / scatter plots)
Save preprocessed data as CSV (preprocessed_data.csv)
End

Result

The Python program successfully simulated IoT sensor readings (temperature and humidity), saved them as raw CSV data, and preprocessed them by handling missing values, removing duplicate records, detecting outliers, and normalizing the readings to a 0–1 range. The cleaned dataset was saved as preprocessed_data.csv, and a plot confirming the cleaned trends was generated as preprocessed_plot.png.

Sample observation table (illustrative):

Raw records collected	50 rows (including 3 missing values and 1 duplicate)
Missing values after cleaning	0
Duplicate rows after cleaning	0
Value range after normalization	0.0 to 1.0
Final preprocessed records	49 rows

Conclusion

In this practical, IoT sensor data was simulated using Python to represent real-world temperature and humidity readings, and a complete preprocessing pipeline was implemented — covering missing value handling, duplicate removal, outlier detection, and normalization. This exercise demonstrates why preprocessing is a mandatory step before IoT data can be used for visualization, analytics, or AI/ML model training, since raw sensor data is rarely clean or analysis-ready in practice.